DevOps Lab Experiment – 6

Continuous Integration with Jenkins: Setting Up a CI Pipeline, Integrating Jenkins with Maven/Gradle, Running Automated Builds and Tests

What is a CI Pipeline?

A **Continuous Integration (CI) Pipeline** automates the process of building, testing, and integrating code changes every time code is committed to the repository.

The CI Pipeline

- Automatically checks out the latest code.
- Compiles the application.
- Runs tests to catch errors early.
- Notifies the team of build/test results.

Why use Jenkins for CI?

Automation: Jenkins automates the build and test cycle, reducing manual intervention.

Immediate Feedback: Developers get rapid notifications of any integration issues.

Extensibility: With hundreds of plugins available, Jenkins can integrate with version control systems, build tools (Maven, Gradle), testing frameworks, and more.

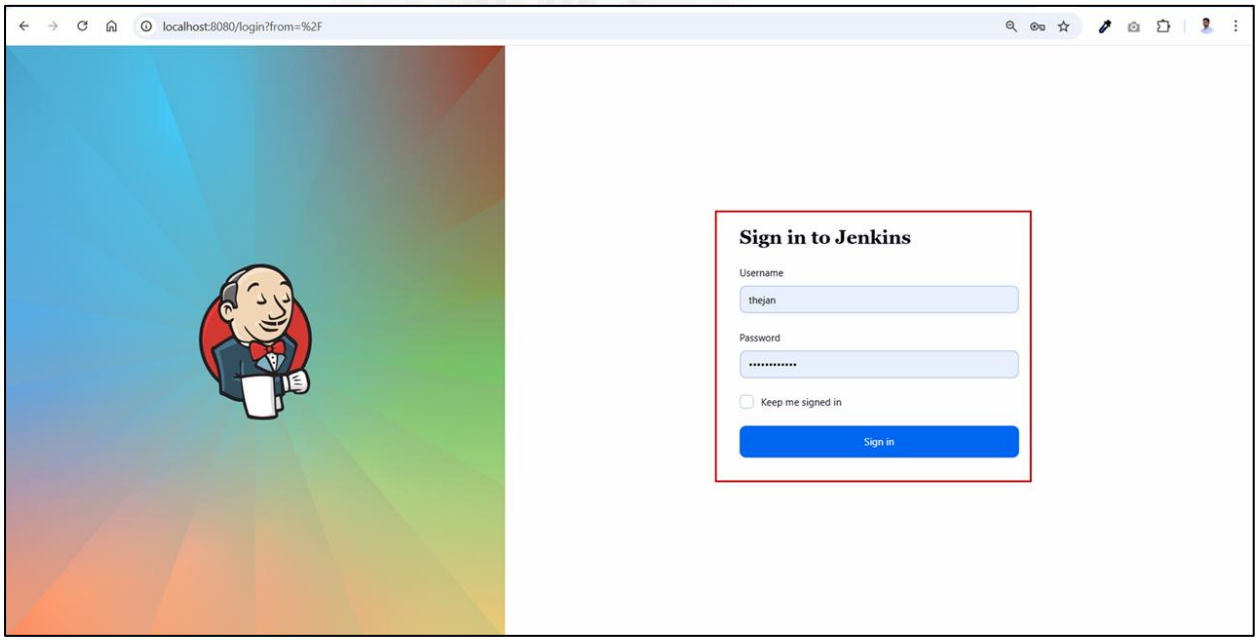
Pipeline as Code: Using Jenkins Pipelines (defined in a Jenkinsfile), you can manage the CI process as part of your source code repository.

Setting Up a CI Pipeline

This section explains how to create a **CI pipeline** as a Freestyle project that integrates with a Maven or Gradle project.

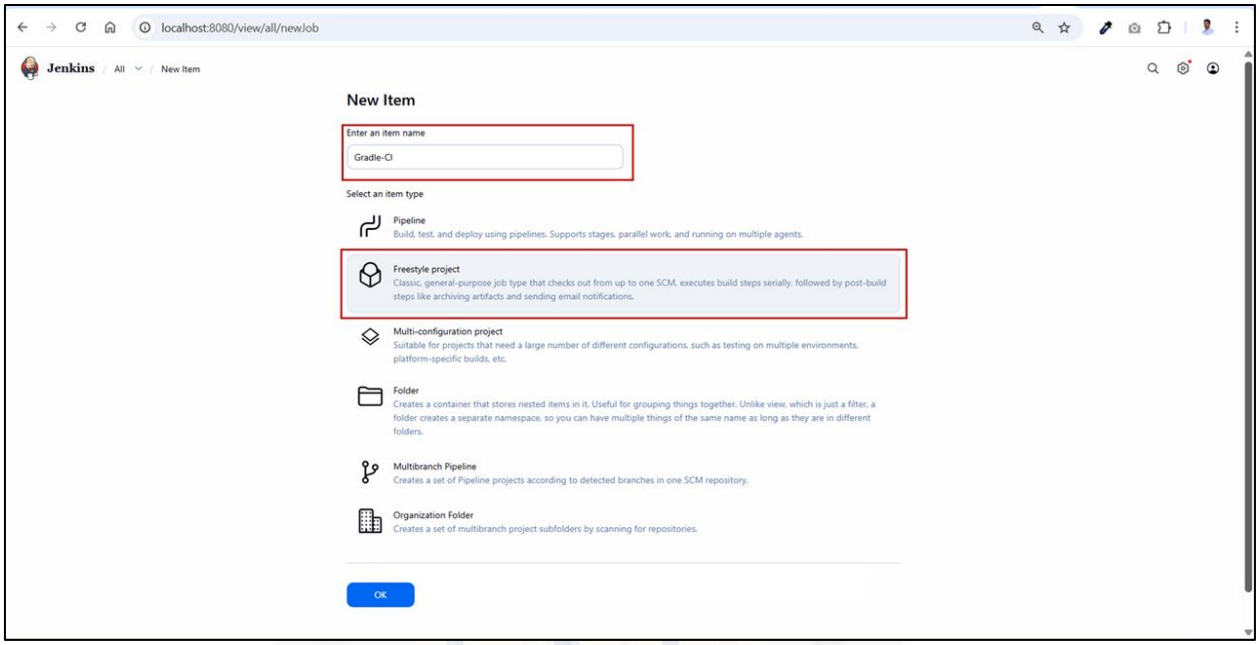
Step 1: Create a New Jenkins Job

1. **Login into Jenkins**
   - Open the web browser and navigate to http://localhost:8080/ and log in with your admin credentials.



Jenkins login page for accessing Jenkins dashboard

2. **Create a New Job**
   - On the Jenkins dashboard, click on **New Item**
   - Enter an item name as Maven-CI (or **Gradle-CI** if you prefer Gradle).
   - Select **Freestyle project**
   - Click **OK**



Creating a New Jenkins Job

Step 2: Configure Source Code Management (SCM)

1. Under **General** section, inside the Description box, enter the description as **CI job to build and test the project using Gradle whenever code changes are pushed to the repository**.
2. Under **Source Code Management** section and select **Git** (if using Git for version control).

   Repositories:

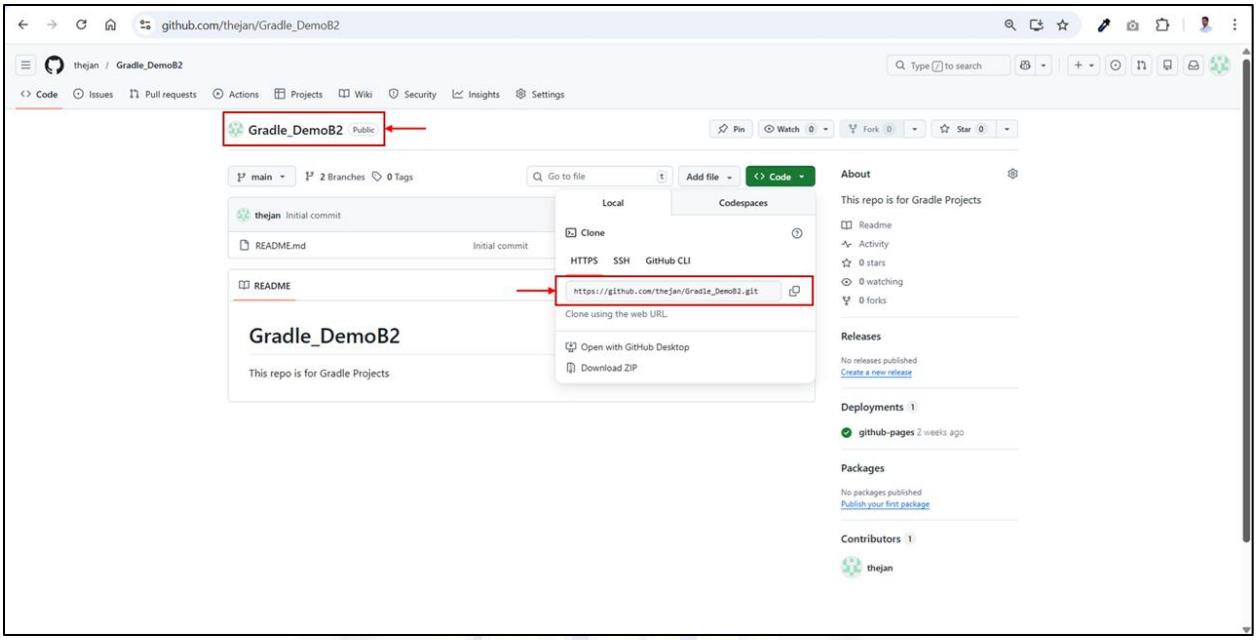
   Repository URL: https://github.com/thejan/Gradle_DemoB2.git

   Credentials: If your repository is **private**, click **Add** to provide the necessary credentials.

   Branches to build:

   Branch specifier: */master

   Repository browser: Auto



GitHub Repository Page

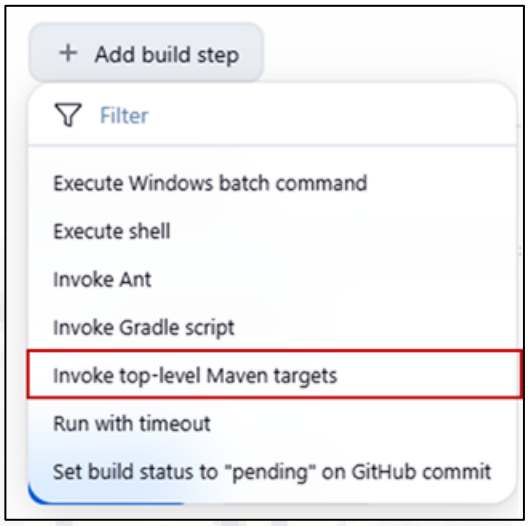
Step 3: Add Build Steps

Scroll down to **Build Steps** section and click **Add build step**.

For integrating **Gradle project**, select **Invoke Grale script** ***(This option might be available if you have installed a Gradle plugin in Jenkins)***

Enter **clean build** in the Taks field.

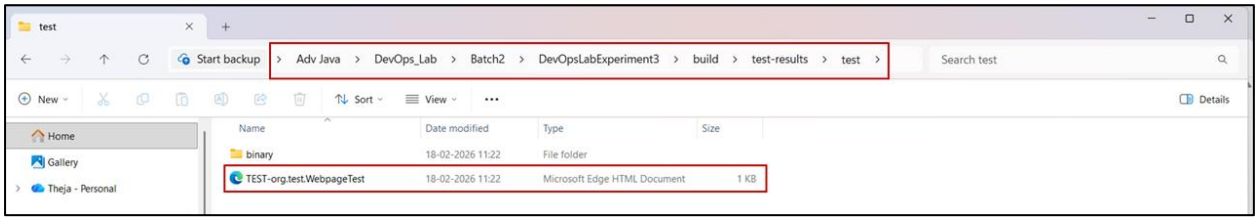
For integrating **Maven project**, select **Top level Maven-targets** and in the Goals field, type **clean package.**



Step 4: Configure Post Build Actions

Scroll down to Post-build Actions and click **post-build action** and select **Publish Junit test result report**.

Test report XMLs: build/test-results/test/*.xml



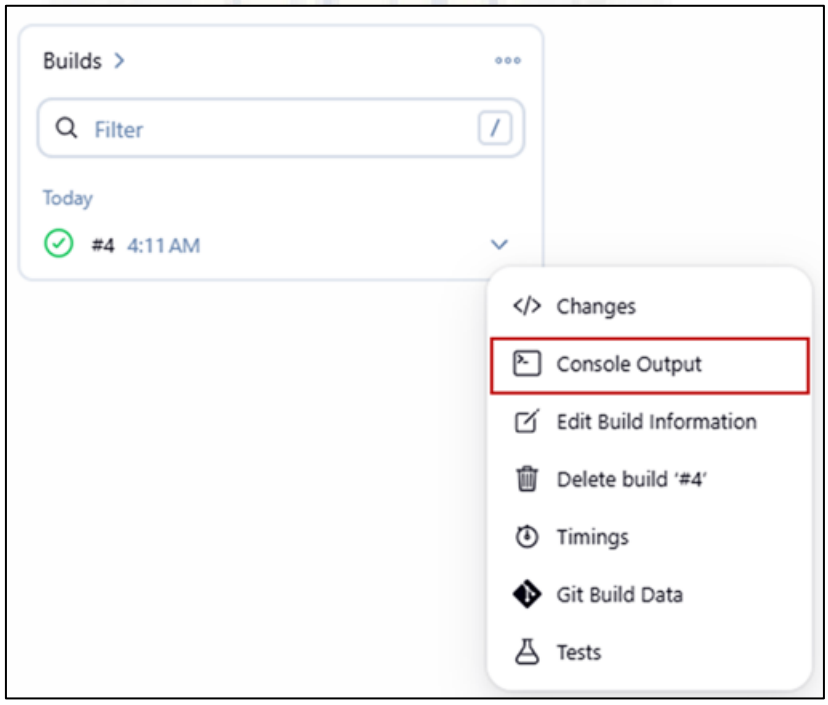
Gradle Test Results Directory and XML Report File

Step 4: Save and Run the Job

Click on **Save** to save the configuration

On the job's main page, click **Build Now** to trigger a build. The build will be added to the build history on the left side.

Click on the **build number** (e.g., #4) and then click **Console Output**.



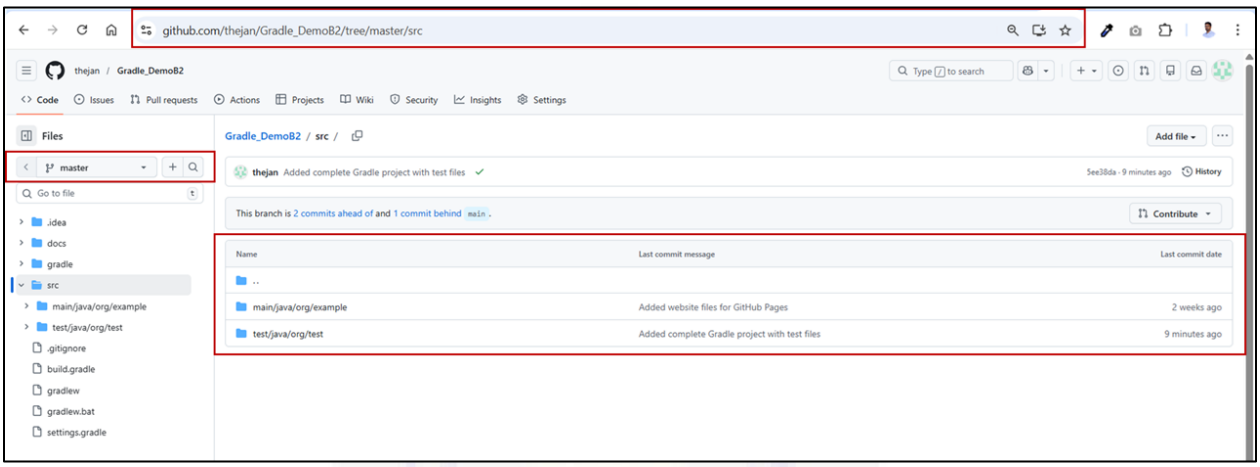
Verify that Jenkins successfully checks out the code, runs the build commands (Maven or Gradle) and executes tests.

Look for **BUILD SUCCESSFUL** or the equivalent output to confirm that the build and tests passed.

```
BUILD SUCCESSFUL in 9s
8 actionable tasks: 8 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.3.0/userguide/configuration_cache_enabling.html
Build step 'Invoke Gradle script' changed build result to SUCCESS
Recording test results
[Checks API] No suitable checks publisher found.
Finished: SUCCESS
```

Successful Jenkins CI Build Showing Gradle Build Completion and Test Results Recorded (Finished: SUCCESS)

NOTE



Open your GitHub repository.

Select the correct branch (master/main as configured in Jenkins).

Verify that the following files are present in the root:

build.gradle

settings.gradle

gradlew / gradlew.bat

Ensure that the **src** folder exists.

Confirm that application code is inside: **src/main/java**

Confirm at least one test file exists inside: **src/test/java**

Verify that your latest changes are committed and pushed.

Ensure the latest commit message is visible on **GitHub**.

After verifying all the above, click **Build Now** in Jenkins.

Step 5: Setting Up a CI Pipeline with Jenkins (Pipeline as Code)

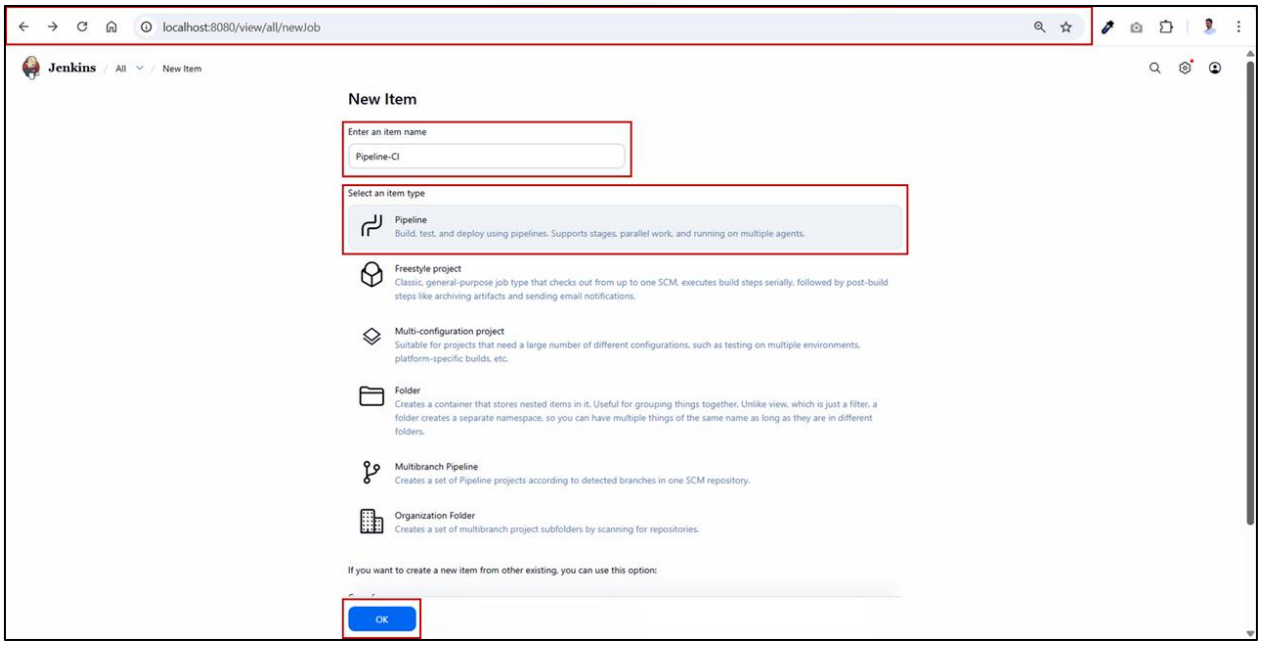
For greater flexibility and version-controlled CI configuration, you can use a **Jenkins Pipeline** defined in a Jenkinsfile.

1. **Create a Pipeline Job**

   Log into **Jenkins** and click on **New Item**.

   Enter an Item Name as **Pipeline-CI**.

   Select **Pipeline** and click **OK**.



Enter the description as **CI Pipeline job to build and test the Gradle project automatically using Jenkinsfile**

Under Pipeline section,

Select **Pipeline script**

Paste the following **Gradle project script** (for DevOpsLabExperiment-3 Gradle Project)

```
pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps {
                git url: 'https://github.com/thejan/Gradle_DemoB2.git', branch: 'master'
            }
        }
        stage('Build') {
            steps {
                bat 'gradlew.bat clean build'
            }
        }
    }

    post {
        always {
            junit '**/build/test-results/test/*.xml'
        }

        success {
            echo 'Build and tests succeeded!'
        }

        failure {
            echo 'Build or tests failed.'
        }
    }
}
```

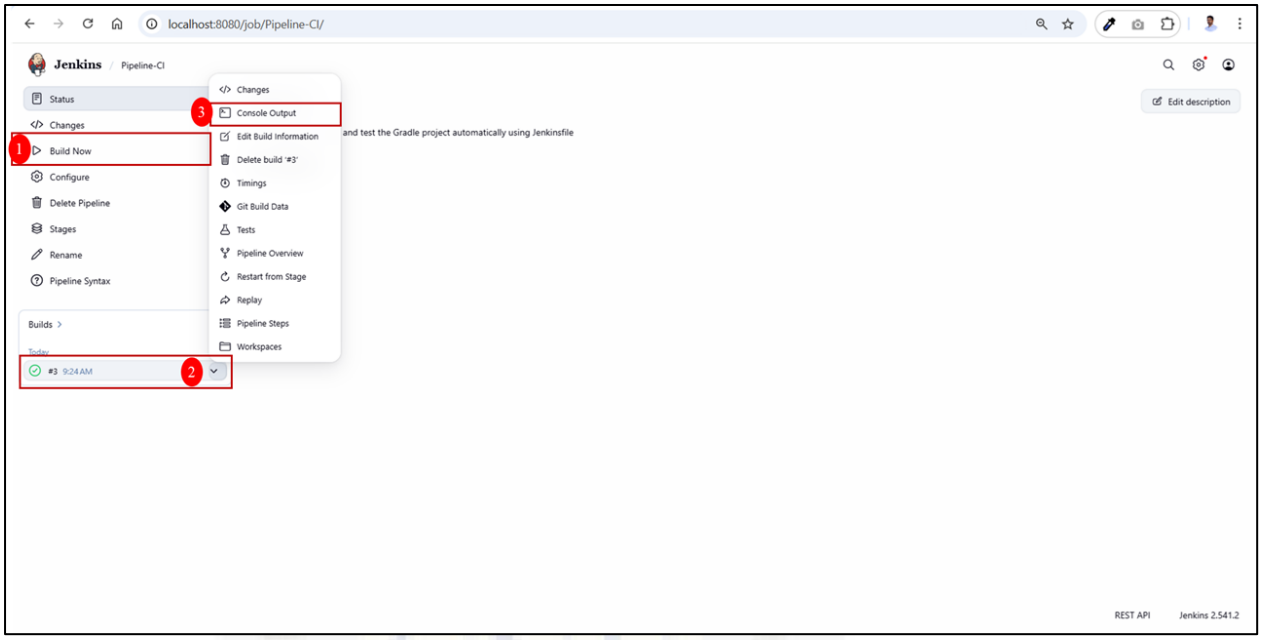
Pipeline Script for Gradle Project

Click on **Save** to save the configuration.

2. **Run the Pipeline**

   On the job's main page, click **Build Now** to trigger a build. The build will be added to the build history on the left side.

   Click on the **build number** (e.g., #3) and then click **Console Output**.



Running the Pipeline

3. Verify the Results

Confirm that each stage (Checkout, Build, Test) executes successfully.

Review the archived test reports to verify that tests have run and passed

```
BUILD SUCCESSFUL in 26s
8 actionable tasks: 7 executed, 1 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.2.0/userguide/configuration_cache_enabling.html
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] junit
Recording test results
[Checks API] No suitable checks publisher found.
[Pipeline] echo
Build and tests succeeded!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Successful Build

If you write a **Pipeline Script** for **Maven project**, refer the following script

```
pipeline {
    agent any

    stages {

        stage('Checkout') {
            steps {
                // Checkout code from GitHub repository
                git url: 'https://github.com/yourusername/your-mavenproject.git', branch: 'main'
            }
        }

        stage('Build') {
            steps {
                // Run Maven build
                bat 'mvn clean package'
            }
        }
    }

post {

        always {
            // Publish JUnit test reports
            junit '**/target/surefire-reports/*.xml'
        }

        success {
            echo 'Build and tests succeeded!'
        }

        failure {
            echo 'Build or tests failed.'
        }
    }
}
```

Pipeline script for Maven Project

Viva Questions

1. What is Jenkins?
2. What is a CI Pipeline?
3. What are the main stages in a CI pipeline?
4. Why is CI important in DevOps?
5. Why is Jenkins widely used for CI?
6. What is meant by Pipeline as Code?
7. What are the two types of Jenkins jobs used in this experiment?
8. What is the difference between a Freestyle project and a Pipeline project?
9. What is a Jenkinsfile?
10. Where is the Jenkinsfile stored?
11. What does agent any mean in a Pipeline?
12. What is a stage in Jenkins Pipeline?
13. What is the purpose of the post block?
14. Why do we use gradlew.bat instead of gradle?
15. What is Jenkins workspace?
16. What is Source Code Management (SCM)?
17. Why do we integrate Jenkins with Git?
18. What is a branch specifier in Jenkins?
19. What happens during the Checkout stage?
20. What is the purpose of mvn clean package?
21. What is the purpose of gradlew clean build?
22. Where are Maven test reports generated?
23. Where are Gradle test reports generated?
24. What is a build artifact?
25. Why do we publish test reports in Jenkins?
26. What is automated testing?
27. What happens if there is no test file in the project?

28. What is Continuous Deployment?
29. What is Continuous Delivery?
30. What is a Jenkins plugin?
31. What is meant by version-controlled CI configuration?
32. Why is Pipeline preferred over Freestyle jobs in real projects?